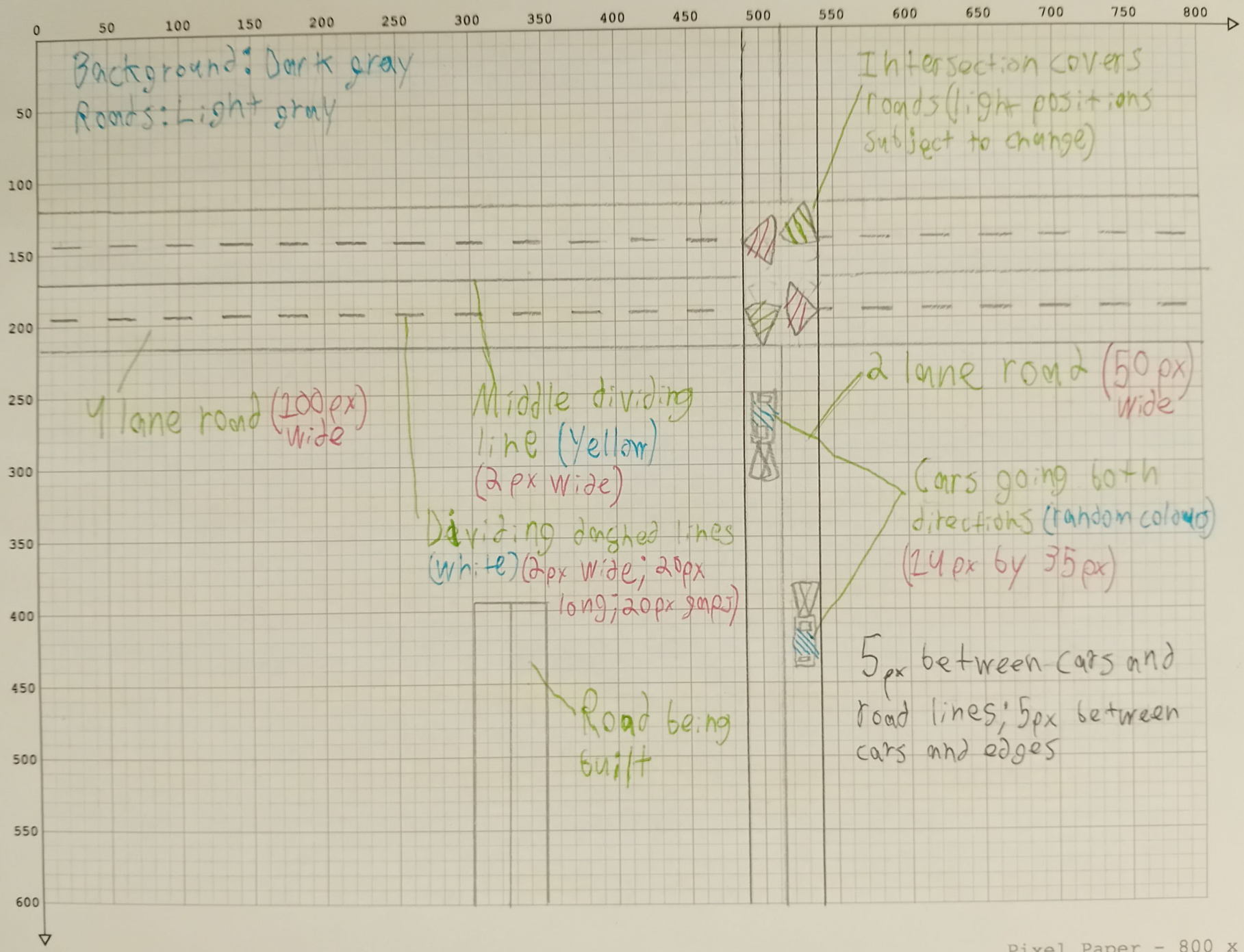


Pixel Paper - 800 x 600
Graph Paper for Screen Coordinates

Assignment 3 Process Work

Pre Planning:
The goal is to create a top-down view of a city (at night) / reigon where the user places down roads. The user can click and drag the mouse to create a road, it will snap to being horisontal or vertical depending on what the path of the mouse drag is closest to (there will be an indicator before letting go). The road will then build (stretch) across the screen from one edge to the other. Once built, cars will spawn and drive straight (no turning) in different directions on different lanes. At every point where roads overlap there will be an intersection to make them look nicer.
Everything beond this point may be cut

Clicking on a road will upgrade it from 2 lanes to 4 lanes, increasing the car spawn rate and speed. Intersections will have lights that cause cars to stop when they turn red. Cars will stop so that they don't hit other cars. Cars will have acceleration speed so they don't stop and start instantly. I may or may not have time to give the cars more detail (windows, headlights, etc...).
If the cars spawn with a random colour, I will have it select from a list of pre-chosen colours.
Optional features; buildings; roads can't be built too close together. middle of road

The roads will have parameters for the direction they're facing; x or y position (not both); Weather or not they're buiding (and their current length so they know when to stop); and the number of lanes, which dictates all other values.

Cars will have a position, speed, and direction and should check every frame if they should stop. They will also have a pointer to the road they're on.
Roads and intersections will have their own arrays, but each road will have an array for its own cars.

Log 1:
I added the ability for users to create roads. Every frame, the program checks if the left mouse button is pressed, down, or released; so that, when the user drags their mouse, there is an arrow indicator, and when released, a road is built. The arrow is drawn using a function. It goes from wherever the mouse was when pressed, to the mouses current position, but 'snapped' to one of the 4 cardinal directions. The direction it snapps to is detemined with some if statements, which allows the function to return that direction as a string that can be passed to a road constructor. When built, the roads stretch from one side of the screen to the other (based on direction) over a few seconds. I spent too much time worrying about implementing that in a way that will make upgrading to 4 lanes easier to add later. I originally wanted to make the gray part of the roads a rectangle, but realized it's easier to use sharpLines. There can be at most 20 roads in the array, which will be more than I need when/if I add distance restrictions.

When I went from 1 road object to an array, it wasn't building more than one road. This was because I didn't add a break statement after the for loop found an empty index, so it just completely filled the array. I should remember this

Log 2:
Cars now spawn on roads after they are finished building. Roads now have a spawn rate (in seconds) and a spawn timer. When the timer reaches the rate* (it increases every frame), it resets and a car is spawned in a random lane. Each lane is its own array of cars; all stored within a greater array (in each road); they can hold up to 20 cars each, which should be enough. For any of these arrays of objects, if they do fill up, the program won't break, it just won't make any more. Cars have a random colour and will move accross the screen (oriented in the correct direction) untill they go offscreen and despawn. I wanted to do this inside of car.Update but realized I have to do it in the main program. The body of the cars are drawn with a quad instead of a rectange so that the origin is always consistent from the car's perspective. It was a bit annoying to deal with so many for loops (because of the arrays) and if statements for each direction, but there was no better way.
* I can add random variation if I want to

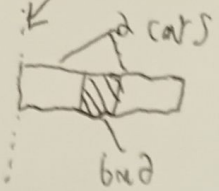
Log 3:
I added the basic functionallity of intersections: they cover overlaps of roads to make them look nicer.

I can change the size of the crosswalks (white outline) at any time to make them look better. After creating a new road, the program checks if it's perpendicular to any other roads and adds intersections at each of those points.

Log 4:
Roads are now prevented from getting too close to eachother. When a new road is created, the program checks if it is too close to any parallel roads, and deletes it if it is. This goes before creating intersections so that intersections don't get created. When this happens, a red X appears where the mouse was pressed for a few seconds.

Log 5:
I added the ability for roads to be upraded from 2 lanes to 4 lanes. Roads now have an upgrade function that just changes the values of parameters. Cars spawn 3.5 times more often, but move 1.5 times faster (leave the screen sooner), and with twice as many lanes to spawn on, each lane should have slightly more cars on average than a lane roads. When a (built) 2 lane road is right clicked, the upgrade function is called, and the other methods of the road handle the rest. I made it right clicks (creating roads is dragging LMB) because that would be significantly easier than inserting it into my left mouse logic. I also made it so that when a road is upgrading, the smaller road is still drawn underneath (this is why I made DrawRoad a function earlier).

fig 1



When first ideating the project, I thought about having a new road build on top of the old one, and then deleting the old one, but soon realized this works much better. Making the dashed white lines animate was slightly annoying (because it has to be different in each direction), but for loops made it easier (with the correct order of hierarchy).
I also implemeted the random variation in spawn rate, doing some calculations to ensure that cars will always have time to get out of the way of the next car (see fig 1).
I realized while making this section that when/if I give the intersections traffic light functionallity, it would be much easier to make them all syncronized (all vertical or horisontal lights green at once) because then I can just tell every road whether or not cars need to stop at any intersections instead of tracking them individually.

In hegnsight, it would have been better if roads were ~~[illegible]~~ built by drawing the whole road, and then masking part of it.

Log 6:
I was completly wrong about that. The only difference is that there would be a global (Game.cs) variable for which direction the lights are facing, but each car would still have to check if its on a road where the lights are red, and then the distance to each intersection.
Cars now have a boolean 'stop' that gets set to true when it gets too close to a car in front of it, or a red light.

Roads do this for every car by comparing the position to the positions of each other car in the lane and each intersection on the road. It checks whether it's in front or behind and what coordinate to use based on the direction. Cars do not stop and start instantly, they accelerate and decelerate. This means that the distance away before it stops is based on how far it will travel before speed = 0 (using kinematics). I also made it so that cars will keep going if they were in the intersection when it turned red.
I fixed a bug where the cars were stuttering because they were accelerating when speed got to 0.

Log 7:
Just to add a little more interactivity, if the spacebar is pressed while hovering over an intersection, its lights will change.
When a light is one second away from changing, the green light will turn yellow. This does nothing, it's just for effect.
Project complete.